МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Book Tracker and Advisor Bot

Архитектура

Митяев О., Попов И., Фомин М.

Санкт-Петербург, 2026

Содержание

1 Требования к технологическому стеку **2**

2 Компонентная диаграмма **2**

2.1 Сервер 2
2.2 Telegram Bot 2
2.3 Пользователь (User) 3
2.4 Администратор (Admin) 3
2.5 Модуль обработки сообщений (MessageHandlerModule) 3
2.6 Модуль трекера книг (TrackerModule) 3
2.7 Модуль предпочтений (PreferencesModule) 4
2.8 Модуль поиска (SearchModule) 4
2.9 Модуль LLM-советника (LLMAdvisorModule) 4
2.10 Модуль данных пользователя (UserModule) 4
2.11 Модуль Healthcheck (HealthCheckModule) 5
2.12 Модуль администрирования (AdminAuthModule) 5
2.13 Модуль списка пользователей (ListUsersModule) 5
2.14 Модуль напоминаний (ZeroMQReminderModule) 5
2.15 API (NoSQL DB) 5
2.16 API (ZeroMQ) 6
2.17 API GigaChat 6
2.18 API Google Books 6
2.19 База данных (MongoDB) 6
2.20 Брокер сообщений (ZeroMQ) 6

Приложение. JSON-schema базы данных **7**

1 Требования к технологическому стеку

Язык программирования: Java 25.
Фреймворк: Spring Framework 7 (Context, Beans, Web, Spring Data MongoDB).
Telegram API: Java Telegram Bot API.
HTTP-клиент: Spring `RestClient`.
Брокер сообщений: ZeroMQ (JeroMQ).
База данных: MongoDB.
Контейнеризация: Docker, Docker Compose.
Сборка: Gradle, упаковка в Fat JAR.
Тестирование: JUnit 5, AssertJ, Mockito.
Документация API: Spring Rest Docs.

2 Компонентная диаграмма

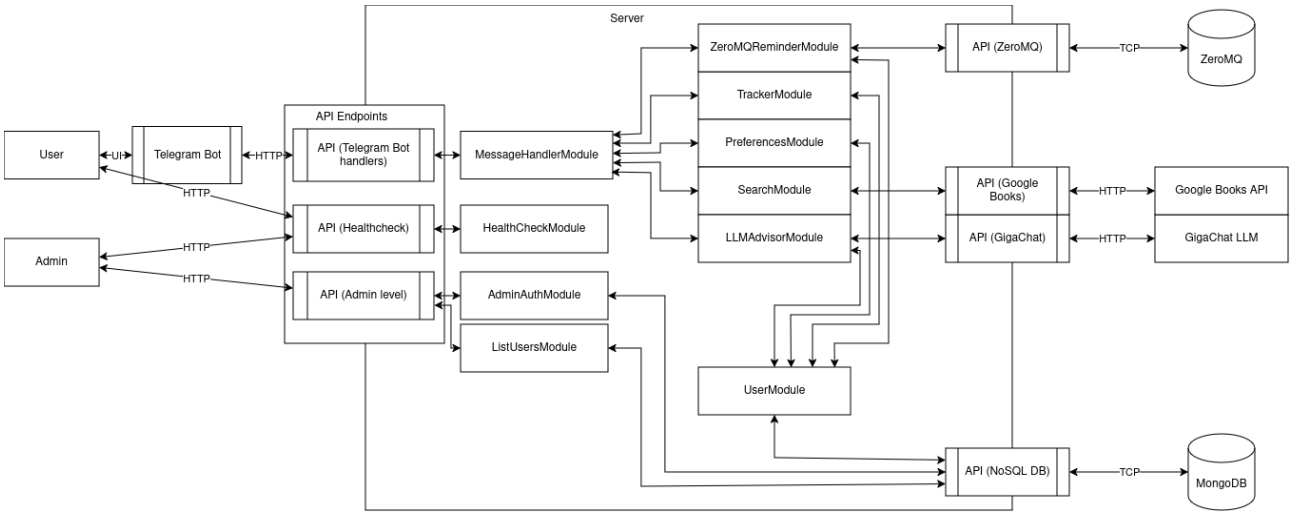

Рис. 1: Компонентная диаграмма системы

2.1 Сервер

Роль: Центральный компонент системы.
Функционал:

- Принимает запросы от Telegram-бота и административные HTTP-запросы.
- Координирует взаимодействие между всеми внутренними модулями.
- Хранит и маршрутизирует бизнес-логику приложения.

2.2 Telegram Bot

Роль: Пользовательский интерфейс.
Функционал:

- Принимает сообщения и команды от пользователей через Telegram.
- Передаёт входящие обновления на сервер через Bot API.
- Отправляет сформированные ответы пользователям.

2.3 Пользователь (User)

Роль: Конечный потребитель сервиса.
Функционал:

- Взаимодействует с системой исключительно через интерфейс Telegram-бота.
- Автоматически идентифицируется по уникальному `chat_id`.
- Может проверить состояние системы через публичный эндпоинт `/healthcheck`.

2.4 Администратор (Admin)

Роль: Пользователь с расширенными правами.
Функционал:

- Осуществляет административные взаимодействия через HTTP API.
- Получает доступ к защищённым эндпоинтам посредством API-ключа, передаваемого в HTTP-заголовке.
- Имеет доступ к полному списку зарегистрированных пользователей через `GET /users`.

2.5 Модуль обработки сообщений (MessageHandlerModule)

Роль: Центральный маршрутизатор команд пользователя.
Функционал:

- Принимает входящие обновления от `API (Telegram Bot handlers)`.
- Разбирает текстовые команды и распределяет задачи по соответствующим модулям.
- Обрабатывает нетекстовые сообщения (фото, стикеры, файлы) и возвращает стандартный ответ с подсказкой.
- Блокирует повторные запросы пользователя, пока предыдущий длительный запрос ещё обрабатывается.

2.6 Модуль трекера книг (TrackerModule)

Роль: Управление читательским списком пользователя.
Функционал:

- Обрабатывает команды `/list`, `/status`, `/progress`, `/remove`.
- Добавляет книги в список через inline-кнопки с выбором статуса (`to_read`, `reading`, `finished`).
- Обновляет статус и прогресс чтения, соблюдая ограничение в 200 книг на пользователя.
- Обрабатывает бонусные команды `/rate`, `/note`, `/quote add`, `/quotes`.
- Обращается к `UserModule` для чтения и записи данных.

2.7 Модуль предпочтений (PreferencesModule)

Роль: Управление литературными предпочтениями пользователя.
Функционал:

- Обрабатывает команды `/preferences add`, `/preferences show`, `/preferences remove`.
- Соблюдает ограничение в 15 предпочтений на пользователя.
- Обращается к `UserModule` для чтения и сохранения предпочтений.

2.8 Модуль поиска (SearchModule)

Роль: Поиск книг через внешний сервис Google Books.
Функционал:

- Обрабатывает команды `/search` и `/random`.
- При вызове `/search` без аргументов формирует запрос на основе предпочтений пользователя из `UserModule`.
- Формирует сообщение-список (до 5 книг) с inline-кнопками пагинации.
- Реализует детальную карточку книги с кнопками «Добавить в список» и «Саммари от LLM».
- При таймауте Google Books API (>5 сек) выполняет 2 повторные попытки с экспоненциальной задержкой.
- Обращается к `API (Google Books)` для выполнения запросов.

2.9 Модуль LLM-советника (LLMAdvisorModule)

Роль: Генерация персонализированных рекомендаций и ответов с помощью GigaChat.
Функционал:

- Обрабатывает команды `/recommend`, `/similar`, `/ask`, а также нажатие кнопки «Саммари от LLM».
- Извлекает историю чтения и предпочтения пользователя из `UserModule` для формирования контекстного промпта.
- Отправляет промежуточное сообщение « *Генерирую ответ, пожалуйста, подождите...*», редактируя его по завершении или при ошибке.
- Соблюдает таймаут 30 секунд; повторные попытки к GigaChat не выполняются.
- Обращается к `API (GigaChat)` для генерации текста.

2.10 Модуль данных пользователя (UserModule)

Роль: Единая точка доступа к пользовательским данным в базе данных.
Функционал:

- Хранит и предоставляет данные пользователей (профиль, список чтения, предпочтения, напоминания).
- Автоматически создаёт запись пользователя при первом обращении по `chat_id`.
- Обращается к `API (NoSQL DB)` для всех операций с базой данных.

2.11 Модуль Healthcheck (HealthCheckModule)

Роль: Системный мониторинг.
Функционал:

- Реализует публичный эндпоинт `GET /healthcheck` без необходимости авторизации.
- Возвращает текущий статус сервера (`{"status": "UP"}`) и список авторов проекта.

2.12 Модуль администрирования (AdminAuthModule)

Роль: Верификация API-ключей администраторов.
Функционал:

- Проверяет наличие и корректность API-ключа в заголовке входящих HTTP-запросов.
- При отсутствии или несовпадении ключа возвращает статус `401 Unauthorized`.
- Обращается к `API (NoSQL DB)` для верификации ключа.

2.13 Модуль списка пользователей (ListUsersModule)

Роль: Получение полного списка зарегистрированных пользователей.
Функционал:

- Обрабатывает запросы на эндпоинт `GET /users`.
- Получает список всех пользователей из базы данных и возвращает его администратору.
- Обращается к `AdminAuthModule` за верификацией ключа и к `API (NoSQL DB)` за данными.

2.14 Модуль напоминаний (ZeroMQReminderModule)

Роль: Асинхронная отправка ежедневных напоминаний о чтении.
Функционал:

- Обрабатывает команды `/remind on <ЧЧ:ММ>` и `/remind off`; сохраняет настройки через `UserModule`.
- Spring-планировщик (`@Scheduled`) ежеминутно проверяет пользователей, которым нужно отправить уведомление.
- Формирует задачи и помещает их в очередь `API (ZeroMQ)` (Producer).
- Отдельный воркер читает очередь (Consumer) и отправляет уведомления через Telegram API.

2.15 API (NoSQL DB)

Роль: Адаптер доступа к MongoDB внутри сервера.
Функционал:

- Предоставляет модулям единый интерфейс для операций с базой данных.
- Взаимодействует с MongoDB по протоколу TCP.

2.16 API (ZeroMQ)

Роль: Адаптер доступа к брокеру сообщений ZeroMQ.
Функционал:

- Абстрагирует отправку и получение сообщений из очереди ZeroMQ.
- Взаимодействует с брокером ZeroMQ по протоколу TCP.

2.17 API GigaChat

Роль: Внешний LLM-провайдер для генерации ответов.
Функционал:

- Принимает POST-запросы с промптами через `RestClient`.
- Возвращает сгенерированный текст ответа.
- Таймаут ожидания — 30 секунд; повторные попытки не выполняются.

2.18 API Google Books

Роль: Внешний сервис поиска метаданных книг.
Функционал:

- Позволяет искать книги по названию, автору, жанру или ключевым словам.
- Возвращает метаданные книги: название, авторы, год издания, описание.
- При таймауте (>5 сек) выполняются 2 повторные попытки с экспоненциальной задержкой.

2.19 База данных (MongoDB)

Роль: Хранение всех данных системы.
Функционал:

- Хранение профилей пользователей, их читательских списков и предпочтений.
- Хранение настроек напоминаний.
- Хранение API-ключей администраторов.

Коллекции:

- `users` — `chatId`; `readingList`; `preferences`; `reminder`.
- `admins` — `apiKeyHash`.

В приложении приведена JSON-schema для хранящихся данных.

2.20 Брокер сообщений (ZeroMQ)

Роль: Асинхронная передача задач на отправку напоминаний.
Функционал:

- Producer (`ZeroMQReminderModule`) помещает задачи в очередь.
- Consumer (`ZeroMQReminderModule`, воркер) вычитывает задачи и отправляет Telegram-уведомления через `RestClient`.

2 Приложение. JSON-schema базы данных

Ниже представлены JSON-schema для коллекций MongoDB.

2.20 Коллекция users

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["chatId"],
  "properties": {
    "_id":    { "type": "string", "description": "MongoDB ObjectId" },
    "chatId": { "type": "string", "description": "
       Telegram chat_id" },
    "readingList": {
      "type": "array",
      "maxItems": 200,
      "items": {
        "type": "object",
        "required": ["bookId", "title", "status"],
        "properties": {
          "bookId":      { "type": "string" },
          "title":       { "type": "string" },
          "author":      { "type": "string" },
          "year":        { "type": "string" },
          "description": { "type": "string" },
          "status": {
            "type": "string",
            "enum": ["to_read", "reading", "finished"]
          },
          "progress": {
            "type": "string",
            "maxLength": 50,
            "description": "                                          (
                                                  reading)"
          },
          "rating": {
            "type": "integer",
            "minimum": 1,
            "maximum": 10,
            "description": "                                          finished
               "
          },
          "note": {
            "type": "string",
            "maxLength": 300,
            "description": "                                (
                                  finished)"
          },
          "quotes": {
            "type": "array",
            "maxItems": 10,
            "items": { "type": "string", "maxLength": 500 }
```

```
          }
        }
      }
    },
    "preferences": {
      "type": "array",
      "maxItems": 15,
      "items": { "type": "string", "maxLength": 50 }
    },
    "reminder": {
      "type": "object",
      "properties": {
        "enabled": { "type": "boolean" },
        "time":    { "type": "string", "pattern": "^([01]\\d|2[0-3])
           :[0-5]\\d$" }
      }
    }
  }
}
```

2.20 Коллекция admins

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["apiKeyHash"],
  "properties": {
    "_id":        { "type": "string", "description": "MongoDB ObjectId"
        },
    "apiKeyHash": { "type": "string", "description": "SHA-256 API-key
       hash" }
  }
}
```